

Identificação do grupo

Nome da disciplina	Segurança da Informação
Nome dos integrantes	Caio Rosa e Iago Koch
Turma	T2ESOFT05NB
Data	Maio de 2026
Link do repositório	https://github.com/caiorosa31/vnuv
Link do sistema publicado	https://caiorosa31.github.io/vnuv/

Descrição resumida do sistema

É um painel para registrar e acompanhar situações envolvendo alunos (contestação de notas, faltas erradas, etc.). As principais funções são: login, cadastro e busca de ocorrências, mudança de status, exclusão, exportação de dados e logs de auditoria.

Tem três tipos de usuário: **Aluno**, **Professor** e **Administrador**.

Regras de negócio percebidas

Quem pode cadastrar ocorrências?

No sistema original, qualquer usuário autenticado podia registrar ocorrências. Na versão corrigida, essa ação ficou restrita a Professores e Administradores.

Quem pode visualizar registros?

Antes, todos viam tudo, incluindo dados sensíveis de terceiros. Na versão corrigida, Alunos veem apenas as próprias ocorrências; Professores e Administradores veem todas, mas dados como CPF e contato completo são exibidos só para o Administrador.

Quem pode alterar ou excluir informações?

No sistema original, qualquer usuário podia fazer as duas coisas. Na versão corrigida, alterar status é permitido para Professor e Administrador; excluir registros é exclusivo do Administrador.

Quais ações deveriam ser restritas?

Exportação de dados, limpeza de logs e restauração da base são ações exclusivas do Administrador. Nenhum usuário final pode alterar o próprio perfil.

Há diferença real entre os perfis no sistema original?

Não. Todos os usuários tinham acesso às mesmas funcionalidades, e qualquer um podia se tornar Administrador com um clique.

Há regras simuladas apenas no front-end?

Sim, todas. Como não há back-end, qualquer restrição pode ser contornada pelo console do navegador editando o localStorage, o que representa a limitação mais crítica do sistema.

Identificação dos ativos

#	Ativo identificado	Por que este ativo tem valor?
1	Dados pessoais dos alunos	Contêm informações de identificação civil e de contato. Sua exposição ou alteração indevida pode causar danos diretos ao aluno, incluindo fraudes e violação da LGPD.
2	Credenciais de acesso	Permitem autenticação no sistema. Se comprometidas, um usuário indesejado pode se passar por qualquer usuário e executar qualquer ação

3	Registros de ocorrências	Contêm histórico acadêmico e situações sensíveis dos alunos. Alteração ou exclusão indevida compromete a integridade dos processos institucionais.
4	Observações internas	Anotações restritas feitas por professores e administradores sobre cada caso.
5	Logs de auditoria	Registram todas as ações realizadas no sistema com timestamps e identificação do usuário. A exclusão ou adulteração impede a reconstrução de incidentes.
6	Dados exportados (arquivo JSON)	A função de exportação gera um arquivo com todas as ocorrências e logs. Na versão original, incluía também as senhas de todos os usuários.

Classificação dos dados e ativos

#	Dado ou ativo	Classificação	Justificativa
1	CPF do aluno	Restrito	Documento de identificação civil protegido pela LGPD. Deve ser acessível apenas ao Administrador, nunca ao próprio aluno por essa interface.
2	Senhas dos usuários	Restrito	Credencial de acesso ao sistema. Sua exposição compromete todos os outros ativos do sistema.

3	E-mail pessoal e telefone do aluno	Confidencial	Dados de contato pessoal. Exposição viabiliza engenharia social, phishing e assédio.
4	Observações internas das ocorrências	Confidencial	Avaliações internas da instituição sobre situações dos alunos. Devem ser visíveis apenas a Professor e Administrador.
5	Logs de auditoria	Confidencial	Contêm rastreamento de todas as ações com identificação de usuários. Revelam padrões de uso sensíveis e devem ser protegidos contra leitura e apagamento por usuários comuns.
6	Nome e matrícula do aluno	Interno	Identificadores acadêmicos necessários para a operação do sistema.
7	Categoria e prioridade da ocorrência	Interno	Dados operacionais de gestão.
8	Status da ocorrência	Interno	Informação de acompanhamento do processo. Pode ser compartilhada com o próprio aluno

Análise dos riscos encontrados

#	Problema identificado	Risco associado	Impacto possível

1	Senhas em texto claro no app.js	Confidencialidade	Qualquer pessoa que abre o código-fonte vê imediatamente os logins e senhas de todos os usuários, incluindo o Administrador.
2	Escalada de privilégio via seletor de perfil	Controle de acesso	Um Aluno pode selecionar o perfil "Administrador" com um único clique, obtendo acesso total ao sistema: exportar dados, excluir ocorrências, apagar logs e restaurar a base.
3	Exportação inclui senhas e localStorage completo	Confidencialidade	Qualquer usuário autenticado podia baixar um arquivo JSON contendo as senhas de todos os usuários, o token de API e uma cópia do armazenamento local.
4	CPF e dados de contato visíveis a todos os perfis	Confidencialidade / LGPD	Um Aluno conseguia visualizar CPF, e-mail e telefone de todos os outros alunos cadastrados no sistema, violando o princípio da necessidade da LGPD e expondo dados pessoais de terceiros.
5	Ausência de validação de entrada	Integridade	Dados incompletos ou mal formatados podiam ser cadastrados (CPF inválido, campos obrigatórios em branco). O checkbox de consentimento LGPD podia ser ignorado, comprometendo a base legal do tratamento de dados
6	Delete/clear/reset sem verificação de perfil	Disponibilidade / Rastreabilidade	Um Aluno autenticado podia excluir qualquer ocorrência, apagar todo o histórico de auditoria e restaurar o sistema ao estado inicial, destruindo dados e eliminando evidências de suas próprias ações.

7	Credenciais listadas e pré-preenchidas no HTML	Confidencialidade	O HTML da página de login exibía explicitamente todos os usuários e senhas, e os campos já vinham preenchidos.
8	Logs acessíveis e apagáveis por qualquer usuário	Rastreabilidade	A seção de logs era visível para Aluno, Professor e Admin. O botão "Limpar logs" estava disponível a todos, permitindo que qualquer usuário eliminasse o histórico de ações.
9	Ausência de proteção contra força bruta no login	Autenticidade	Sem limite de tentativas, qualquer atacante podia testar combinações ilimitadas de senha de forma automatizada sem qualquer impedimento do sistema.

Melhorias implementadas no código

#	Melhoria implementada	Arquivo alterado	O que mudou no sistema?

1	Substituição de senhas por hashes	app.js	O campo password foi removido do array USERS e substituído por passwordHash. Senhas em texto claro não existem mais no código.
2	Sessão sem credenciais no localStorage	app.js	A função saveSession() desestrutura o objeto do usuário e descarta os campos password e passwordHash antes de persistir no localStorage. A sessão salva contém apenas nome, e-mail, perfil e matrícula.
3	Controle de acesso baseado em perfil (RBAC)	app.js	Criação do mapa PERMISSIONS com ações permitidas por perfil e da função can como ponto único de verificação.
4	Restrição do seletor de perfil ao Administrador	app.js / index.html	O elemento #roleSwitchSection é ocultado para perfis sem a permissão change_role . A função changeRole() verifica can("change_role") antes de qualquer alteração.
5	Delegação de eventos em vez de onclick inline	app.js	Os botões da tabela usam atributos data-action e data-id . Um único listener em occurrencesTable trata todos os cliques, eliminando event handlers inline.

6	Mascaramento de dados pessoais por perfil	app.js	Criadas as funções <code>maskCpf()</code> , <code>maskEmail()</code> e <code>maskPhone()</code> . Na <code>render()</code> , dados sensíveis são exibidos completos apenas para ADMIN; os demais perfis veem versões mascaradas.
7	Filtro de ocorrências para o perfil Aluno	app.js	Na função <code>render()</code> , quando o perfil é ALUNO, o array de ocorrências é filtrado para exibir apenas os registros onde <code>studentEmail</code> ou <code>studentId</code> correspondem ao usuário autenticado.
8	Exportação segura restrita ao Administrador	app.js	A função <code>exportEverything()</code> verifica <code>can("export")</code> . O JSON exportado não contém mais o array <code>USERS</code> , hashes de senhas, token de API nem cópia do <code>localStorage</code> , só ocorrências e logs.
9	Validação de entrada no formulário	app.js	Criada a função <code>validateOccurrenceForm()</code> que verifica campos obrigatórios, formato de matrícula, formato de CPF, formato de e-mail e marcação do checkbox de consentimento LGPD antes de salvar.
10	Proteção contra força bruta no login	app.js	Adicionado o objeto <code>loginAttempts</code> com contador e timestamp de bloqueio. Após 5 tentativas falhas consecutivas, o sistema bloqueia novas tentativas por 30 segundos. A mensagem de erro informa quantas tentativas restam.

12	Remoção de credenciais pré-preenchidas e listadas	index.html	Removidos os atributos value=" professor@faculdade.local " e value="123456" dos campos de login. O bloco .
13	Logs de auditoria restritos ao Administrador	app.js / index.html	A seção #auditSection é ocultada para perfis sem a permissão clear_logs . O botão "Limpar logs" também é restrito ao ADMIN via verificação can() .

Relação entre problema, risco, controle e justificativa

#	Item analisado	Risco associado	Controle ou melhoria proposta	Foi implem entado
1	Senhas em texto claro no código-fonte	Comprometimento imediato de todas as contas ao visualizar o código	Armazenar hashes; verificar via Web Crypto API no login	Sim
2	Senha salva na sessão do localStorage	Credencial persistida no navegador após o logout	saveSession() descarta campos de credencial antes de salvar	Sim
3	Escalada de privilégio via seletor de perfil	Aluno obtém permissões de Administrador com um clique	Seletor visível/funcional apenas para ADMIN; can("change_role") na função	Sim

5	CPF/e-mail/telefone visíveis a todos os perfis	Violação da LGPD; exposição de dados pessoais a perfis não autorizados	Mascaramento por perfil; visualização completa apenas para ADMIN	Sim
6	Aluno visualiza ocorrências de outros alunos	Exposição de situações acadêmicas de terceiros	Filtro em render() por e-mail/matricula do usuário autenticado	Sim
7	Observação interna visível ao Aluno	Aluno acessa avaliações internas restritas sobre si mesmo	Campo exibido apenas para perfis com permissão view_internal_notes	Sim
8	Exportação inclui senhas e localStorage completo	Vazamento massivo de credenciais e dados sensíveis	Export restrito ao ADMIN; payload sem credenciais, token ou localStorage	Sim
9	Delete/clearLogs/reset sem verificação de perfil	Aluno destrói dados e apaga rastros de auditoria	Verificação can() no início de cada função; log de tentativas negadas	Sim
10	Formulário sem validação de dados	Dados inválidos, incompletos ou sem consentimento salvos no sistema	Função validateOccurrenceForm() com regras de formato e obrigatoriedade	Sim
11	Credenciais pré-preenchidas e listadas no HTML	Qualquer visitante acessa o sistema sem qualquer esforço	Campos sem valor pré-definido; demo-users exibe apenas e-mails, sem senhas	Sim

12	Ausência de proteção contra força bruta	Tentativas ilimitadas de adivinhação de senha sem bloqueio	Bloqueio em memória por 30s após 5 tentativas falhas consecutivas	Parcial
13	Logs acessíveis e limpáveis por qualquer usuário	Rastreabilidade comprometida; usuário apaga rastros de ações	Seção de logs e botão de limpar restritos ao ADMIN	Sim
14	Ações destrutivas sem confirmação do usuário	Clique acidental ou mal-intencionado destrói dados sem chance de cancelar	confirm() obrigatório antes de excluir, limpar logs e resetar	Sim
15	Logs de auditoria apagáveis pelo Administrador	Mesmo o Admin pode encobrir suas próprias ações	Logs em banco de dados append-only com restrição de exclusão	Depen banco dados

Justificativa técnica das decisões

Item 1 e 2 — Hash SHA-256 para senhas e sessão sem credenciais Problema: As senhas ficavam em texto puro no código, e o objeto completo do usuário (incluindo senha) era salvo no localStorage. Risco: Qualquer pessoa com acesso ao navegador podia ler as senhas via F12. Controle: Senhas substituídas por hashes SHA-256. A sessão passa a salvar apenas os dados necessários, descartando credenciais. Limitação: SHA-256 sem salt é vulnerável a senhas fracas. A solução ideal seria Argon2 ou bcrypt no servidor.

Item 3 — Controle de acesso por perfil (RBAC) Problema: Qualquer usuário podia alterar o próprio perfil, inclusive virar Administrador. Risco: Controle de acesso completamente quebrado (OWASP A01:2021). Controle: Sistema de permissões declarativo com verificação centralizada. O seletor de perfil só aparece para quem tem permissão explícita (exclusivo do Admin). Limitação: Controle apenas no front-end, contornável pelo console do navegador.

Itens 5, 6 e 7 — Mascaramento de dados pessoais Problema: CPF, e-mail, telefone e observações internas eram visíveis para todos os perfis. Risco: Violação da LGPD — exposição de dados pessoais de terceiros sem necessidade. Controle: Funções de mascaramento aplicadas por perfil. Alunos veem apenas as próprias ocorrências e não acessam observações internas.

Item 8 — Exportação segura Problema: A exportação incluía senhas e tokens e estava disponível a qualquer usuário. Risco: Vazamento massivo de dados com um único clique. Controle: Exportação restrita a quem tem permissão. O arquivo exportado contém apenas ocorrências e logs, sem credenciais.

Item 10 — Validação de entrada Problema: O formulário aceitava qualquer dado e ignorava o checkbox de consentimento da LGPD. Risco: Dados corrompidos e ausência de base legal para o tratamento. Controle: Validação com regras de formato, campos obrigatórios e verificação de consentimento antes de salvar.

Item 12 — Proteção contra força bruta Problema: O login não limitava tentativas. Risco: Ataques automatizados de dicionário sem impedimento. Controle: Após 5 tentativas falhas, o acesso é bloqueado por 30 segundos. Limitação: O bloqueio é perdido ao recarregar a página. Proteção real exige controle por IP no servidor.

Limitações da solução entregue

Limitações por ser somente front-end Todos os controles vivem no navegador. Qualquer pessoa com conhecimento básico de JavaScript pode abrir o console (F12) e contornar acessos, manipular o localStorage ou executar funções protegidas diretamente.

Limitações por não haver banco de dados Os dados ficam no localStorage de cada dispositivo — ou seja, cada navegador tem seus próprios dados, o usuário pode apagar tudo a qualquer momento e não há backup nem controle de integridade centralizado.

Limitações por não haver autenticação real Os hashes SHA-256 estão no próprio código-fonte, visível a qualquer um. Com senhas fracas como "123456" e "admin", um atacante recupera as senhas em segundos com ferramentas comuns como o hashcat.

Limitações por não haver autorização no servidor O RBAC impede ações pela interface, mas não protege contra manipulação direta. Executar `changeRole("ADMIN")` no console contorna todas as verificações.

Limitações dos logs de auditoria Os logs ficam no localStorage e podem ser lidos ou apagados diretamente por qualquer pessoa com acesso ao navegador. Não há assinatura

nem hash de integridade — não é possível provar que não foram adulterados, e não há garantia de não-repúdio.

Limitações da persistência local O `localStorage` tem capacidade limitada (5–10 MB), não sincroniza entre dispositivos, não é criptografado por padrão e pode ser limpo pelo próprio usuário.

Limitações da versão publicada (GitHub Pages) O GitHub Pages serve via HTTPS, mas não permite configurar cabeçalhos de segurança como CSP, HSTS ou X-Frame-Options via servidor. A CSP implementada por meta tag oferece proteção parcial. Além disso, o código-fonte fica publicamente acessível no repositório.

Conclusão do grupo

O sistema original não poderia ser usado em produção: senhas expostas no código, qualquer usuário podendo virar Admin, dados pessoais visíveis a todos, exportação revelando credenciais e ausência de validação de entrada ou controle LGPD.

Após as melhorias, o sistema tornou-se adequado para uso didático: senhas com hash, controle de acesso por perfil, mascaramento de dados sensíveis, mitigação de XSS e CSP ativa. Ainda assim, não seria seguro para dados reais — toda a autorização pode ser contornada pelo console, os hashes são fracos e públicos, e não há como garantir conformidade com a LGPD sem back-end.

Para se tornar um sistema real, seria necessário:

1. Criar um servidor com API REST autenticada;
2. Usar JWT ou sessão server-side com cookie `HttpOnly`;
3. Substituir o `localStorage` por banco de dados com criptografia;
4. Adotar Argon2 ou `bcrypt` com salt para senhas;
5. Implementar logs de auditoria imutáveis no banco;
6. Configurar HTTPS, HSTS e cabeçalhos de segurança no servidor;
7. Realizar análise de conformidade LGPD com apoio jurídico.

Segurança não é uma funcionalidade adicionada depois, mas uma propriedade que deve ser considerada desde as primeiras decisões de arquitetura. Escolhas simples onde armazenar credenciais, quem pode executar cada ação, quais dados exibir para cada perfil têm impacto direto e mensurável sobre a confidencialidade, integridade e disponibilidade do sistema.

